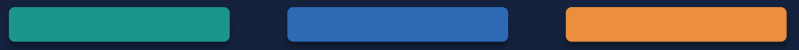
# Python Libraries, NumPy, Pandas & Matplotlib



# Learning goals

What we will cover

- Understand what a Python library is and why programmers use libraries.
- Learn how to create and import your own reusable Python module/package.
- Use NumPy for arrays, calculations, indexing and summary statistics.
- Use Pandas to load, inspect, clean, select, filter and summarize a CSV dataset.
- Use Matplotlib to visualize the data through graphs and charts.
- Practice commands on Bengaluru_House_Data.csv

# What is a Python library?

- A library is a collection of reusable code: functions, classes and tools.
- It saves time by avoiding the need to write common operations from scratch.
- Import a library, then call the functionality it provides.
- Examples: NumPy (numerical computing), Pandas (tabular data), Matplotlib (plots).
- In everyday Python, a single .py file is often called a module; a package groups modules.

# Why use libraries?

Less repetition • More reliability • Faster development

- Productivity: use tested operations instead of rebuilding them.
- Readability: concise commands communicate intent.
- Performance: optimized libraries can run heavy operations efficiently.
- Consistency: standard tools make code easier to share and maintain.
- Example: NumPy can add every element in two arrays with one expression.

# Create your own library: a module

A module is a reusable Python file

Python

```
# 1) Create a file named mytools.py

def square(number):
    return number * number

def greet(name):
    return f"Hello, {name}!"
```

# Import and use your module

Run from the same folder

Python

```
# 2) In main.py
import mytools

print(mytools.square(5))
print(mytools.greet("Amit"))

# Alternative: import a specific function
from mytools import square
print(square(8))
```

# Create a small package

Organize multiple modules

```
my_package/
    __init__.py
    math_tools.py
    text_tools.py

# math_tools.py
def add(a, b):
    return a + b

# Use it
from my_package.math_tools import add
print(add(2, 3))
```

# Meet the dataset

Bengaluru_House_Data.csv

- Dataset size: 13,320 records × 9 columns.
- Each row describes a property listing.
- Columns include area_type, availability, location, size, society, total_sqft, bath, balcony and price.
- The price column is expressed in the dataset’s price unit; do not assume it is a per-square-foot value.
- We will use the same file for both NumPy and Pandas examples.

# Load the CSV with Pandas

Start with a DataFrame

```python
import pandas as pd

df = pd.read_csv("Bengaluru_House_Data.csv")

print(df.shape)
print(df.head())
```

# NumPy: the basics

Fast numerical arrays

Python

```
import numpy as np

prices = np.array([39.07, 120.00, 62.00, 75.00])

print(prices)
print(prices.mean())
print(prices.max())
print(prices.min())
```

# NumPy: index, slice and calculate

Array positions start at zero

Python

```
prices = np.array([39.07, 120, 62, 75])

print(prices[0])       # first value
print(prices[1:3])     # index 1 up to (not incl.) 3
print(prices * 2)      # element-wise
print(prices > 70)     # Boolean mask
```

# NumPy on the housing dataset

Convert a numeric column to an array

Python

```
# Convert Pandas column to NumPy
price_array = df["price"].to_numpy()

print(price_array.shape)
print(np.nanmean(price_array))
print(np.nanmin(price_array))
print(np.nanmax(price_array))
```

# Pandas: inspect your data

First commands after reading a CSV

Python

```
df.head()        # first 5 rows
df.tail(3)       # last 3 rows
df.shape         # (rows, columns)
df.columns       # column names
df.info()        # types and non-null counts
df.describe()    # numeric summary
```

# Pandas: select and filter

Choose columns or rows that match a condition

Python

```
# One column → Seriesdf["location"]

# Several columns → DataFrame
df[["location", "size", "price"]]

# Houses with price below 50
df[df["price"] < 50]

# Listings in one location
df[df["location"] == "Uttarahalli"]
```

# Pandas: missing values and cleaning

Inspect before modifying

Python

```
df.isna().sum()       # missing count per column

# Remove rows missing price
clean_df = df.dropna(subset=["price"])

# Fill missing balcony with smedian
df["balcony"] = df["balcony"].fillna(
    df["balcony"].median()
)
```

# Pandas: summarize and group

Answer simple questions from the data

Python

```
# Average listed price
df["price"].mean()

# Number of listings by area type
df["area_type"].value_counts()

# Average price by area type
df.groupby("area_type")["price"].mean()

# Highest priced rows
df.nlargest(5, "price")[["location", "size", "price"]]
```

# Matplotlib: turn data into visuals

A beginner-friendly plotting library

Python

- **Line chart**: Shows trends over time.
- **Bar chart:** Compares categories.
- **Histogram:** Shows the distribution of values.
- **Scatter plot:** Shows the relationship between two variables.

# Matplotlib: turn data into visuals

A beginner-friendly plotting library

Python

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4]
sales = [10, 14, 12, 18]

plt.plot(days, sales)
plt.title("Sales over time")
plt.xlabel("Day")
plt.ylabel("Sales")
plt.show()
```

# Matplotlib: essential commands

Choose a chart and label it clearly

Python

```
plt.plot(x, y)          # line chart
plt.bar(labels, values) # bar chart
plt.hist(values)        # histogram
plt.scatter(x, y)       # scatter plot

plt.title("Chart title")
plt.xlabel("X-axis label")
plt.ylabel("Y-axis label")
plt.grid(True)
plt.show()
# Optional: plt.savefig("chart.png")
```

# Plot 1: house-price distribution

Histogram using the Bengaluru housing dataset

Python

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Bengaluru_House_Data.csv")

plt.hist(df["price"].dropna(), bins=30)
plt.title("Distribution of house prices")
plt.xlabel("Price (dataset units)")
plt.ylabel("Number of listings")
plt.show()
```

# Plot 2: average price by area type

Pandas grouping + Matplotlib bar chart

Python

```
avg_price = (
    df.groupby("area_type")["price"]
      .mean()
      .sort_values()
)

avg_price.plot(kind="bar")
plt.title("Average price by area type")
plt.xlabel("Area type")
plt.ylabel("Average price")
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()
```

# Plot 3: house size vs. price

Scatter plot to explore a relationship

Python

```
plot_df = df[["total_sqft", "price"]].copy()
plot_df["total_sqft"] = pd.to_numeric(plot_df["total_sqft"], errors="coerce")
plot_df = plot_df.dropna()

plt.scatter(plot_df["total_sqft"], plot_df["price"],
            alpha=0.4)
plt.title("House size vs. price")
plt.xlabel("Total area (sq ft)")
plt.ylabel("Price (dataset units)")
plt.show()
```

# Putting the libraries together

A simple analysis workflow

Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("Bengaluru_House_Data.csv")
prices = df["price"].to_numpy()
print("Average price:", np.nanmean(prices))

df.groupby("area_type")["price"].mean().plot(kind="bar")
plt.ylabel("Average price")
plt.tight_layout()
plt.show()
```

# Mini experiments

Hands-on tasks using the Bengaluru housing dataset

## Complete these experiments in a Jupyter Notebook or Google Colab.

### 1 | Your own module

Create myhousing.py with price_category(price): “Budget” if price < 50, “Mid-range” if 50–100, otherwise “Premium”. Import and test with 39, 75 and 120.

### 2 | Pandas: inspect

Load the CSV. Show shape, head(10), column names and unique locations. Select location, size and price into a new DataFrame.

### 3 | Pandas: filter/group

Filter price < 50. Group by area_type and calculate mean and median price. Display the five highest-priced listings.

### 4 | Clean data

Count missing values. Convert total_sqft to numeric using errors='coerce'. Keep rows with valid total_sqft and price; report remaining rows.

### 5 | NumPy experiment

Convert price to a NumPy array. Calculate nanmean, nanmedian, nanmin and nanmax. Count listings above the median using a Boolean mask.

### 6 | Matplotlib charts

Create a 30-bin price histogram, mean-price bar chart by area_type, and scatter plot of numeric total_sqft vs price. Add titles and axis labels.

### 7 | Combine all tools

Use Pandas to load/group, NumPy to calculate average price, and Matplotlib to plot mean price by area_type. Save as housing_prices.png.

# Quick recap

What each tool does—and how they fit together

| | |
|---|---|
| **Python libraries** | Reusable collections of functions and tools that reduce repeated work. |
| **Your module/package** | Write reusable functions in .py files and import them into another script or notebook. |
| **NumPy** | Arrays, indexing, vectorized operations, Boolean masks and numerical summaries such as mean and median. |
| **Pandas** | Read CSV files into DataFrames; inspect, select, filter, clean, group and summarize tabular data. |
| **Matplotlib** | Create line, bar, histogram and scatter plots; label charts and display them with plt.show(). |
| **Complete workflow** | Pandas loads/prepares data → NumPy computes → Matplotlib visualizes → interpret the result. |

# Thank you

Questions & practice

## Start small. Run each command. Inspect the output.

**Dataset: Bengaluru_House_Data.csv**